LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF320803)
MINGGU 8 ACARA 21

SEMESTER 3



Tugas :
Dokumentasi endpoint

Disusun Oleh:
Uaryl Engno Laurenca (E41250317)

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

A. PENDAHULUAN

Pada Acara 17 sampai 20, API Sistem Informasi Akademik berkembang dari CRUD sederhana menjadi API dengan format response seragam, autentikasi JWT, dan penanganan error berlapis. Setiap acara menghasilkan endpoint baru, tetapi pengetahuan tentang cara memakainya masih tersebar di kode dan di kepala pengembangnya. Orang lain, misalnya tim frontend atau mobile, tidak bisa memakai API ini tanpa membaca kode sumber.

Acara 21 menyelesaikan masalah tersebut dengan menyusun dokumentasi endpoint. Dokumentasi yang baik mempercepat integrasi, mengurangi komunikasi bolak-balik antara tim backend dan frontend, dan menjadi rujukan resmi ketika terjadi perbedaan pemahaman tentang perilaku suatu endpoint. Dokumentasi disusun dalam dua bentuk: berkas Markdown (`docs/API_DOCUMENTATION.md`) dan dokumentasi di Postman (Description, Examples, dan View Documentation). Acara ini tidak mengubah kode PHP sama sekali.

B. TUJUAN PRAKTIKUM

1. Memahami pentingnya dokumentasi API dan elemen dokumentasi endpoint yang baik.
2. Menyusun struktur folder dokumentasi (docs/) pada proyek.
3. Mendokumentasikan peta Router, yaitu pemetaan resource URL ke Controller.
4. Mendokumentasikan endpoint autentikasi (register dan login) dan endpoint mahasiswa, lengkap dengan header, body request, serta contoh response sukses dan gagal.
5. Memanfaatkan fitur Postman (Description, Examples, View Documentation) untuk dokumentasi.
6. Menyusun tabel ringkasan seluruh endpoint beserta Controller penanganannya di bagian awal dokumen (tugas mandiri).

C. DASAR TEORI

3.1 Pentingnya Dokumentasi API

Dokumentasi yang tidak jelas dan sering usang adalah salah satu keluhan terbesar developer ketika mengintegrasikan API pihak lain. Dokumentasi API yang baik mempercepat integrasi, mengurangi kebutuhan komunikasi antara tim backend dan frontend, serta menjadi rujukan resmi untuk perilaku endpoint.

3.2 Elemen Dokumentasi Endpoint yang Baik

- Method HTTP dan URL endpoint
- Header yang dibutuhkan (misalnya Content-Type dan Authorization)
- Struktur body request beserta tipe data tiap field
- Contoh response untuk skenario sukses dan gagal, lengkap dengan kode status
- Keterangan autentikasi yang dibutuhkan (publik atau terproteksi)

3.3 Dokumentasi API melalui Postman

Selain sebagai alat pengujian, Postman menyediakan fitur untuk menuliskan deskripsi pada setiap request dan Collection. Postman juga dapat menghasilkan tampilan dokumentasi (View Documentation) secara otomatis dari request dan contoh response yang sudah disimpan. Dengan begitu, proses dokumentasi menyatu dengan proses pengujian yang sudah dilakukan sejak Acara 19.

3.4 Format Dokumentasi Markdown

Markdown adalah format teks ringan yang dapat dibaca langsung sebagai teks biasa dan dirender menjadi tampilan rapi (misalnya di GitHub). Berkas Markdown dapat disimpan di dalam repositori bersama kode, sehingga perubahan dokumentasi bisa dilacak oleh Git seperti perubahan kode.

D. HASIL DAN PEMBAHASAN

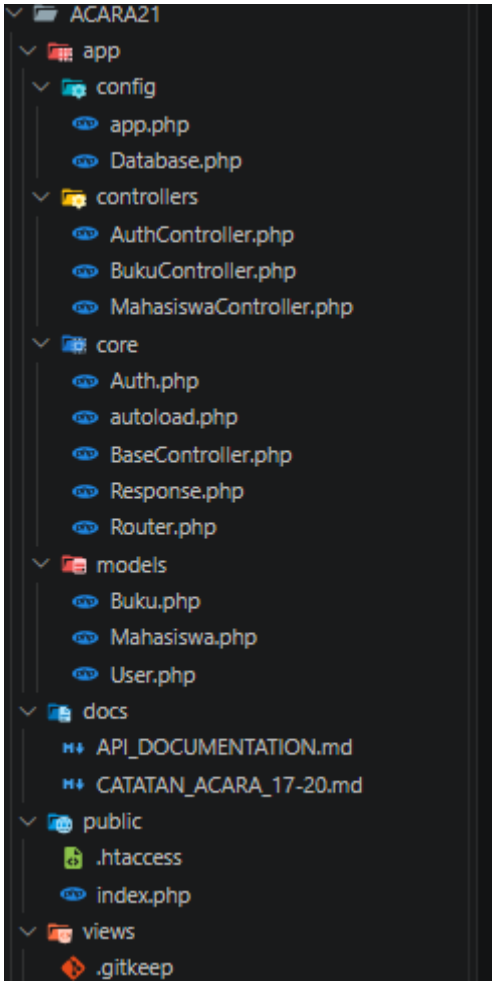
4.1 Alat dan Bahan

XAMPP (Apache, PHP, MySQL), Postman, Visual Studio Code, Git dan GitHub, dan web browser.

4.2 Gambaran Perubahan

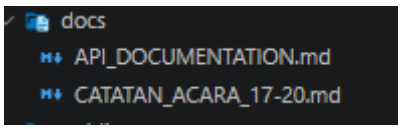
Dokumentasi disusun berdasarkan seluruh Controller yang sudah dibangun: AuthController (register dan login), MahasiswaController (index, store, update, destroy), dan BukuController (pola sama dengan Mahasiswa).

- docs/API_DOCUMENTATION.md : baru, dokumentasi seluruh endpoint
- docs/CATATAN_ACARA_17-20.md : arsip dokumentasi lama (diagram alur dan diskusi JWT)
- Collection Postman : diperbarui dengan Description, Examples, dan dua request Buku tambahan
- Kode PHP, db_workshop.sql, dan Environment Postman : tidak berubah dari Acara 20



[SCREENSHOT 1: struktur folder proyek di VS Code, terlihat folder docs]

4.3 Langkah 1: Membuat Struktur Folder Dokumentasi
Dibuat folder docs/ di dalam proyek sistem-informasi-akademik dengan berkas API_DOCUMENTATION.md. Dokumentasi lama dari Acara 17 sampai 20 dipindahkan ke docs/CATATAN_ACARA_17-20.md sebagai arsip agar tidak ada dua dokumen utama yang saling bertentangan.



[SCREENSHOT 2: isi folder docs di VS Code]

4.4 Langkah 2: Mendokumentasikan Peta Router
Bagian awal dokumen menyertakan tabel pemetaan Router agar pembaca memahami alur permintaan sebelum membaca detail tiap endpoint.

- /mahasiswa : MahasiswaController, CRUD data mahasiswa (index, store, update, destroy)
- /buku : BukuController, CRUD data buku
- /login : AuthController::login, autentikasi pengguna dan menghasilkan token
- /register : AuthController::register, pendaftaran pengguna baru

Pemetaan HTTP method pada resource CRUD: GET ke index(), POST ke store(), PUT ke update($id), dan DELETE ke destroy($id). ID dikirim lewat query string ?id=. Method lain dibalas 405, dan resource yang tidak dikenal dibalas 404.



```
## 2. Peta Router

`Router::dispatch()` membaca segmen terakhir URL sebagai nama resource, lalu memanggil Controller yang sesuai.

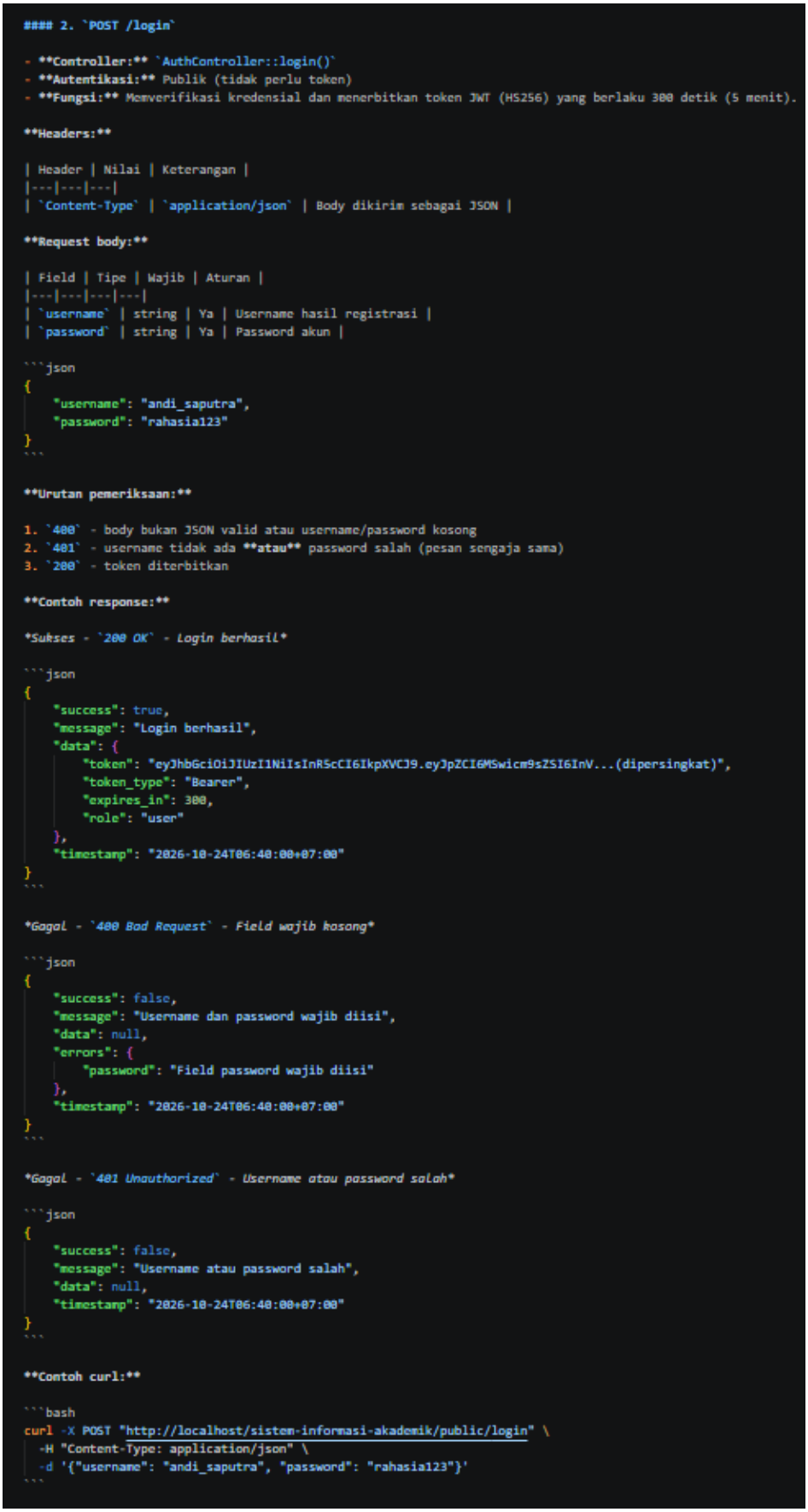
| Resource (URL) | Controller | Keterangan |
|---|---|---|
| `/mahasiswa` | `MahasiswaController` | CRUD data mahasiswa (`index` / `store` / `update` / `destroy`, dipilih dari HTTP method) |
| `/buku` | `BukuController` | CRUD data buku (pola sama dengan mahasiswa) |
| `/login` | `AuthController::login` | Autentikasi pengguna, menghasilkan token. Hanya menerima POST |
| `/register` | `AuthController::register` | Pendaftaran pengguna baru. Hanya menerima POST |

Pemetaan HTTP method pada resource CRUD: `GET` -> `index()`, `POST` -> `store()`, `PUT` -> `update($id)`, `DELETE` -> `destroy($id)`. ID dikirim lewat query string `?id=`. Method lain dibalas
`405`; resource yang tidak ada dibalas `404`.

```
Client -> public/index.php (Front Controller) -> Router::dispatch()
            -> Controller (Auth::requireAuth() bila terproteksi, validasi)
            -> Model (PDO + prepared statement) -> MySQL
            -> Response::json() -> Client
```
```

[SCREENSHOT 3: bagian "Peta Router" pada API_DOCUMENTATION.md]

4.5 Langkah 3: Dokumentasi Endpoint Autentikasi
Endpoint POST /register dan POST /login didokumentasikan dengan pola yang sama. Contoh untuk POST /login:
Controller: AuthController::login()
Autentikasi: publik
Header: Content-Type: application/json
Request body: username (string, wajib) dan password (string, wajib)
Urutan pemeriksaan: 400 jika body bukan JSON valid atau field kosong, 401 jika username tidak ada atau password salah (pesan sengaja sama), 200 jika berhasil.
Response 200 OK berisi data.token, token_type "Bearer", expires_in 300, dan role. Response gagal yang didokumentasikan: 401 "Username atau password salah" dan 400 untuk field kosong. Untuk POST /register didokumentasikan response 201 serta kegagalan 400, 409 (username sudah dipakai), 422 (email tidak valid), dan 405 (method selain POST).



2. `POST /login`

- **Controller:** `AuthController::login()`
- **Autentikasi:** Publik (tidak perlu token)
- **Fungsi:** Memverifikasi kredensial dan menerbitkan token JWT (HS256) yang berlaku 300 detik (5 menit).

Headers:

Header	Nilai	Keterangan
`Content-Type`	`application/json`	Body dikirim sebagai JSON

Request body:

Field	Tipe	Wajib	Aturan
`username`	string	Ya	Username hasil registrasi
`password`	string	Ya	Password akun

```json
{
    "username": "andi_saputra",
    "password": "rahasia123"
}
```

Urutan pemeriksaan:

1. `400` - body bukan JSON valid atau username/password kosong
2. `401` - username tidak ada **atau** password salah (pesan sengaja sama)
3. `200` - token diterbitkan

Contoh response:

Sukses - `200 OK` - Login berhasil

```json
{
    "success": true,
    "message": "Login berhasil",
    "data": {
        "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6InV...(dipersingkat)",
        "token_type": "Bearer",
        "expires_in": 300,
        "role": "user"
    },
    "timestamp": "2026-10-24T06:40:00+07:00"
}
```

Gagal - `400 Bad Request` - Field wajib kosong

```json
{
    "success": false,
    "message": "Username dan password wajib diisi",
    "data": null,
    "errors": {
        "password": "Field password wajib diisi"
    },
    "timestamp": "2026-10-24T06:40:00+07:00"
}
```

Gagal - `401 Unauthorized` - Username atau password salah

```json
{
    "success": false,
    "message": "Username atau password salah",
    "data": null,
    "timestamp": "2026-10-24T06:40:00+07:00"
}
```

Contoh curl:

```bash
curl -X POST "http://localhost/sistem-informasi-akademik/public/login" \
  -H "Content-Type: application/json" \
  -d '{"username": "andi_saputra", "password": "rahasia123"}'
```

[SCREENSHOT 4: bagian dokumentasi POST /login pada API_DOCUMENTATION.md]

1. `POST /register`

- **Controller:** `AuthController::register()`
- **Autentikasi:** Publik (tidak perlu token)
- **Fungsi:** Mendaftarkan pengguna baru. Role selalu `user` (input `role` dari client diabaikan).

Headers:

Header	Nilai	Keterangan
`Content-Type`	`application/json`	Body dikirim sebagai JSON

Request body:

Field	Tipe	Wajib	Aturan
`username`	string	Ya	3-50 karakter; hanya huruf, angka, titik, garis bawah; harus unik
`email`	string	Ya	Format email valid (`filter_var`), maks 100 karakter, harus unik
`password`	string	Ya	8-72 karakter. Disimpan sebagai hash, tidak pernah dikembalikan

```json
{
    "username": "andi_saputra",
    "email": "andi@example.com",
    "password": "rahasia123"
}
```

Urutan pemeriksaan:

1. `400` - body bukan JSON valid atau ada field yang kosong
2. `422` - format data tidak valid (email, panjang password, karakter username)
3. `409` - username atau email sudah terdaftar
4. `201` - password di-hash dengan `password_hash()`, data disimpan

Contoh response:

Sukses - `201 Created` - Registrasi berhasil

```json
{
    "success": true,
    "message": "Registrasi berhasil",
    "data": {
        "id": 1,
        "username": "andi_saputra",
        "email": "andi@example.com",
        "role": "user"
    },
    "timestamp": "2026-10-24T06:40:00+07:00"
}
```

Gagal - `400 Bad Request` - Field wajib kosong

```json
{
    "success": false,
    "message": "Username, email, dan password wajib diisi",
    "data": null,
    "errors": {
        "password": "Field password wajib diisi"
    },
    "timestamp": "2026-10-24T06:40:00+07:00"
}
```

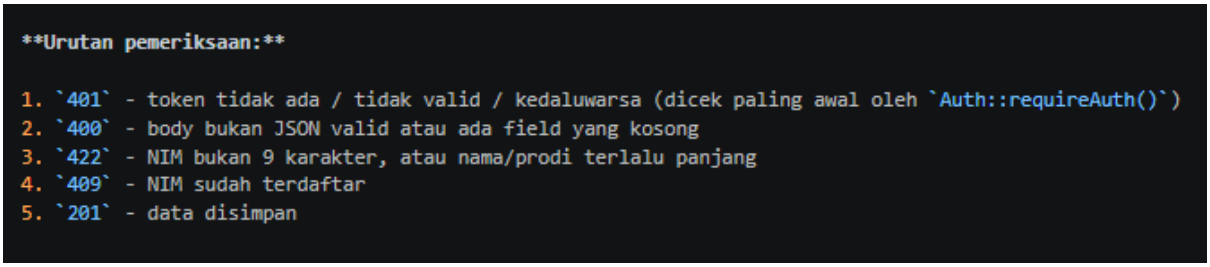
[SCREENSHOT 5: bagian dokumentasi POST /register]

4.6 Langkah 4: Dokumentasi Endpoint Mahasiswa

Pola yang sama diulang untuk setiap method pada MahasiswaController. Hal yang ditekankan: operasi tulis (store, update, destroy) memerlukan header Authorization: Bearer <token> karena memanggil Auth::requireAuth(), sedangkan GET bersifat publik. Setiap endpoint memuat tabel field request body beserta aturannya (misalnya nim tepat 9 karakter dan unik), urutan pemeriksaan, contoh response sukses dan gagal, serta contoh perintah curl.

Contoh urutan pemeriksaan POST /mahasiswa: 401 (token), 400 (body atau field kosong), 422 (format NIM), 409 (NIM duplikat), lalu 201.

Endpoint yang didokumentasikan: GET /mahasiswa, GET /mahasiswa?id={id}, POST /mahasiswa, PUT /mahasiswa?id={id}, dan DELETE /mahasiswa?id={id}. Resource Buku didokumentasikan dengan pola yang sama.



[SCREENSHOT 6: bagian dokumentasi POST /mahasiswa, terlihat keterangan token dan tabel field]



Gagal - `401 Unauthorized` - Tanpa token

(header `WWW-Authenticate: Bearer`)

```json
{
    "success": false,
    "message": "Token tidak ditemukan",
    "data": null,
    "timestamp": "2026-10-24T06:40:00+07:00"
}
```

Gagal - `401 Unauthorized` - Token tidak valid atau kedaluwarsa

(header `WWW-Authenticate: Bearer error="invalid_token"`)

```json
{
    "success": false,
    "message": "Token tidak valid atau kedaluwarsa",
    "data": null,
    "timestamp": "2026-10-24T06:40:00+07:00"
}
```

Gagal - `409 Conflict` - NIM sudah terdaftar

```json
{
    "success": false,
    "message": "NIM sudah terdaftar",
    "data": null,
    "errors": {
        "nim": "NIM sudah dipakai mahasiswa lain"
    },
    "timestamp": "2026-10-24T06:40:00+07:00"
}
```

Gagal - `422 Unprocessable Entity` - NIM bukan 9 karakter

```json
{
    "success": false,
    "message": "Data tidak lolos validasi",
    "data": null,
    "errors": {
        "nim": "Format NIM tidak valid, harus 9 digit"
    },
    "timestamp": "2026-10-24T06:40:00+07:00"
}
```

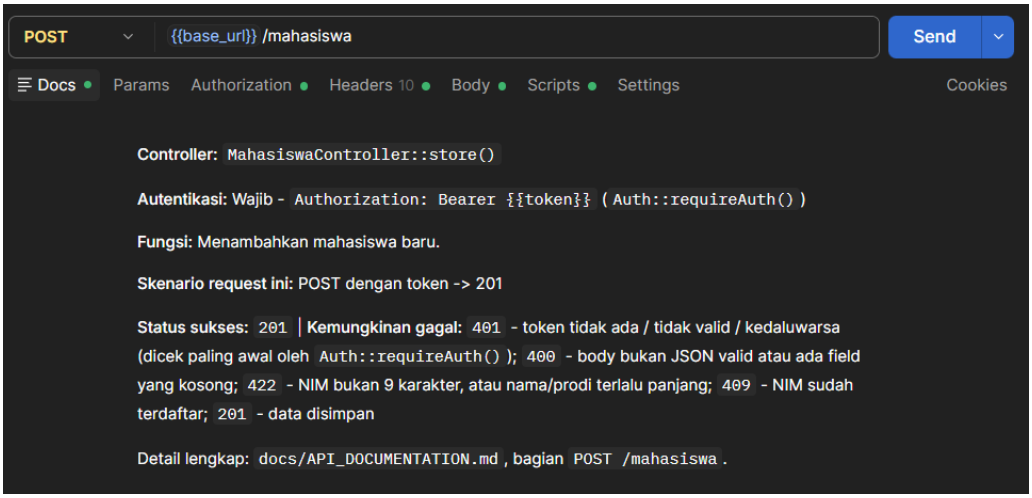
[SCREENSHOT 7: contoh response gagal (401, 409, atau 422) pada dokumentasi]

4.7 Penyusunan Contoh Response

Seluruh contoh response pada dokumen dihasilkan dari eksekusi nyata kode, bukan ditulis tangan, sehingga isinya pasti sesuai perilaku API. Nilai yang berubah tiap eksekusi (timestamp dan created_at) diseragamkan, dan token dipersingkat agar mudah dibaca.

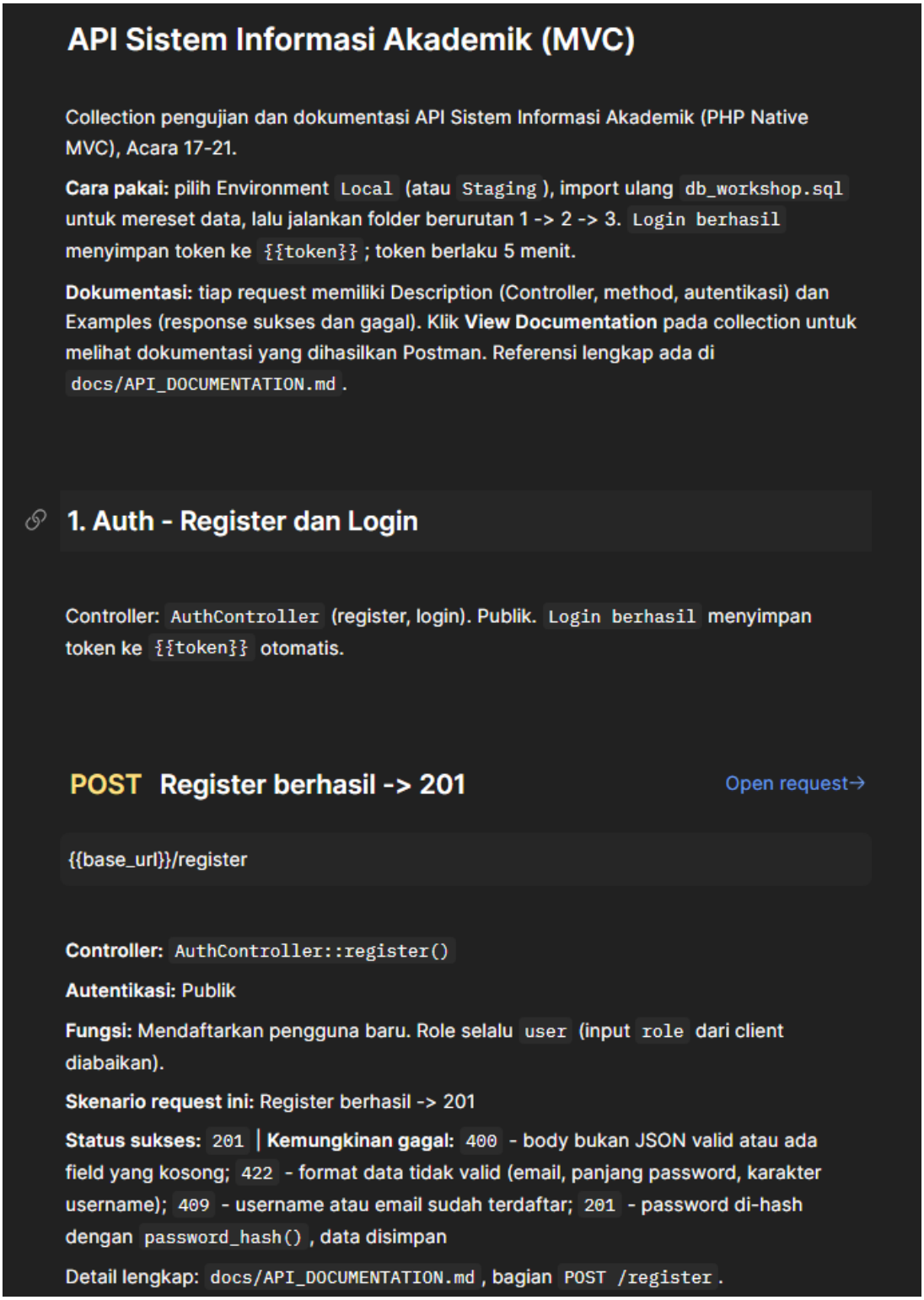
4.8 Langkah 5: Dokumentasi melalui Fitur Postman

1. Collection dari Acara 19 dibuka. Pada setiap request, kolom Description diisi dengan nama Controller dan method yang menangani, kebutuhan autentikasi, dan daftar status code.



[SCREENSHOT 8: Description pada salah satu request di Postman]

2. Contoh response sukses dan gagal disimpan pada tab Examples. Request pertama pada tiap endpoint memuat seluruh contohnya, misalnya POST /mahasiswa memiliki contoh 201, 400, 401, 409, dan 422.
3. Dua request Buku ditambahkan (GET dengan id dan PUT dengan id) agar setiap endpoint yang didokumentasikan memiliki request di Collection.
4. Pada Collection dipilih View Documentation, dan tautan atau tampilannya disimpan sebagai lampiran laporan.



[SCREENSHOT 11: tampilan View Documentation dari Collection]

[LAMPIRAN: tautan dokumentasi Postman (https://documenter.getpostman.com/view/58712611/2sBYHXPxe9)]

4.9 Pengujian Kesesuaian Dokumentasi

Agar dokumentasi tidak usang, dilakukan pengecekan:

1. Seluruh request pada Collection dijalankan berurutan dengan Collection Runner. Hasil yang diharapkan: semua test lulus, yang menunjukkan perilaku API masih sama dengan yang didokumentasikan.

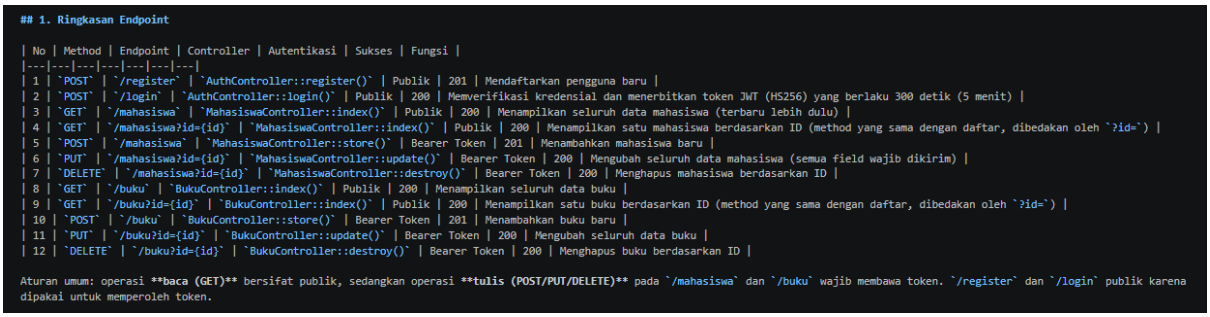
2. Tabel ringkasan dicocokkan dengan peta Router dan Controller pada kode: setiap resource ada di tabel, setiap Controller::method yang disebut benar-benar ada, dan status proteksi token pada dokumen sesuai pemanggilan Auth::requireAuth() di kode.
3. Seluruh blok contoh JSON pada dokumen diperiksa dan semuanya valid.

4.10 Tugas Mandiri: Tabel Ringkasan Endpoint

Bagian pertama dokumen berisi tabel ringkasan seluruh endpoint beserta Controller penanganannya:

1. POST /register : AuthController::register() : publik : 201
2. POST /login : AuthController::login() : publik : 200
3. GET /mahasiswa : MahasiswaController::index() : publik : 200
4. GET /mahasiswa?id={id} : MahasiswaController::index() : publik : 200
5. POST /mahasiswa : MahasiswaController::store() : Bearer Token : 201
6. PUT /mahasiswa?id={id} : MahasiswaController::update() : Bearer Token : 200
7. DELETE /mahasiswa?id={id} : MahasiswaController::destroy() : Bearer Token : 200
8. GET /buku : BukuController::index() : publik : 200
9. GET /buku?id={id} : BukuController::index() : publik : 200
10. POST /buku : BukuController::store() : Bearer Token : 201
11. PUT /buku?id={id} : BukuController::update() : Bearer Token : 200
12. DELETE /buku?id={id} : BukuController::destroy() : Bearer Token : 200

Tabel ini diletakkan di bagian awal agar pembaca langsung mendapat gambaran seluruh API sebelum membaca detailnya.



1. Ringkasan Endpoint

No	Method	Endpoint	Controller	Autentikasi	Sukses	Fungsi
1	`POST`	`/register`	`AuthController::register()`	Publik	201	Mendaftarkan pengguna baru
2	`POST`	`/login`	`AuthController::login()`	Publik	200	Memverifikasi kredensial dan menerbitkan token JWT (HS256) yang berlaku 300 detik (5 menit)
3	`GET`	`/mahasiswa`	`MahasiswaController::index()`	Publik	200	Menampilkan seluruh data mahasiswa (terbaru lebih dulu)
4	`GET`	`/mahasiswa?id={id}`	`MahasiswaController::index()`	Publik	200	Menampilkan satu mahasiswa berdasarkan ID (method yang sama dengan daftar, dibedakan oleh `?id=`)
5	`POST`	`/mahasiswa`	`MahasiswaController::store()`	Bearer Token	201	Menambahkan mahasiswa baru
6	`PUT`	`/mahasiswa?id={id}`	`MahasiswaController::update()`	Bearer Token	200	Mengubah seluruh data mahasiswa (semua field wajib dikirim)
7	`DELETE`	`/mahasiswa?id={id}`	`MahasiswaController::destroy()`	Bearer Token	200	Menghapus mahasiswa berdasarkan ID
8	`GET`	`/buku`	`BukuController::index()`	Publik	200	Menampilkan seluruh data buku
9	`GET`	`/buku?id={id}`	`BukuController::index()`	Publik	200	Menampilkan satu buku berdasarkan ID (method yang sama dengan daftar, dibedakan oleh `?id=`)
10	`POST`	`/buku`	`BukuController::store()`	Bearer Token	201	Menambahkan buku baru
11	`PUT`	`/buku?id={id}`	`BukuController::update()`	Bearer Token	200	Mengubah seluruh data buku
12	`DELETE`	`/buku?id={id}`	`BukuController::destroy()`	Bearer Token	200	Menghapus buku berdasarkan ID

Aturan umum: operasi **baca (GET)** bersifat publik, sedangkan operasi **tulis (POST/PUT/DELETE)** pada `/mahasiswa` dan `/buku` wajib membawa token. `/register` dan `/login` publik karena dipakai untuk memperoleh token.

[SCREENSHOT 13: tabel "Ringkasan Endpoint" di bagian awal API_DOCUMENTATION.md]

4.11 Pembahasan

1. Dokumentasi sebagai kontrak. Dengan dokumentasi, pihak lain dapat memakai API tanpa membaca kode. Tabel field, urutan pemeriksaan, dan contoh response membuat perilaku endpoint dapat diprediksi, termasuk kondisi gagalnya.
2. Contoh dihasilkan dari kode, bukan ditulis tangan. Contoh yang ditulis tangan mudah menyimpang dari perilaku sebenarnya, misalnya pesan error atau status code

yang berubah setelah refactor. Karena contoh diambil dari eksekusi nyata, dokumen dan kode selalu sejalan pada saat penulisan.

3. Dokumentasi harus dirawat. Dokumentasi yang usang lebih berbahaya daripada tidak ada dokumentasi, karena menyesatkan pembacanya. Karena itu pengecekan kesesuaian (Collection Runner dan pencocokan dengan kode) dilakukan, dan pembaruan dokumentasi menjadi bagian dari setiap perubahan Controller atau Router.
4. Satu sumber rujukan. Dokumentasi lama dipindahkan ke arsip sehingga hanya ada satu dokumen utama, yaitu API_DOCUMENTATION.md.
5. Keterangan autentikasi di setiap endpoint. Membedakan endpoint publik dan terproteksi sangat penting. Pembaca langsung tahu bahwa GET tidak memerlukan token, sedangkan POST, PUT, dan DELETE memerlukannya.
6. Dua bentuk dokumentasi saling melengkapi. Markdown berada di repositori dan ikut versi kode, sedangkan dokumentasi Postman bersifat interaktif karena pembaca dapat langsung mencoba request. Keduanya bersumber dari contoh response yang sama.
7. Catatan aturan NIM. Aturan di kode adalah NIM tepat 9 karakter, tidak harus angka, sedangkan pesan error menyebut "9 digit". Dokumen mencatat aturan sesuai perilaku kode.
8. Kode 403 belum dipakai. Belum ada pembatasan berdasarkan role, sehingga status 403 tidak muncul pada dokumentasi.

E. KESIMPULAN

Dokumentasi API berhasil disusun dalam file `docs/API_DOCUMENTATION.md` yang mencakup peta Router, format response, status code, mekanisme penggunaan token, serta detail 12 endpoint autentikasi, mahasiswa, dan buku. Setiap endpoint dilengkapi informasi Controller, kebutuhan autentikasi, header, tabel field request body, urutan pemeriksaan, serta contoh response sukses dan gagal. Collection Postman juga dilengkapi Description dan Examples pada setiap request sehingga dapat menghasilkan tampilan View Documentation. Tugas mandiri terpenuhi dengan adanya tabel ringkasan seluruh endpoint beserta Controller penanganannya di bagian awal dokumen. Seluruh dokumentasi telah diverifikasi melalui pengujian Collection Postman dan pencocokan dengan implementasi kode API untuk memastikan kesesuaian serta keakuratan informasi sebagai referensi penggunaan API.

.